

이벤트 소싱과 CQRS 적용 시행 착오

장기판 상태 저장에 이벤트 소싱이 어울리지 않는 이유

발표 순서

어떤 마음으로 발표를 들으면 좋을까?

1. 이벤트 소싱이란 무엇인가? 이론 살펴보기

2. CQRS는 무엇이며 궁합이 좋은 이유?

3. 장기 미션 적용 시행착오 소개

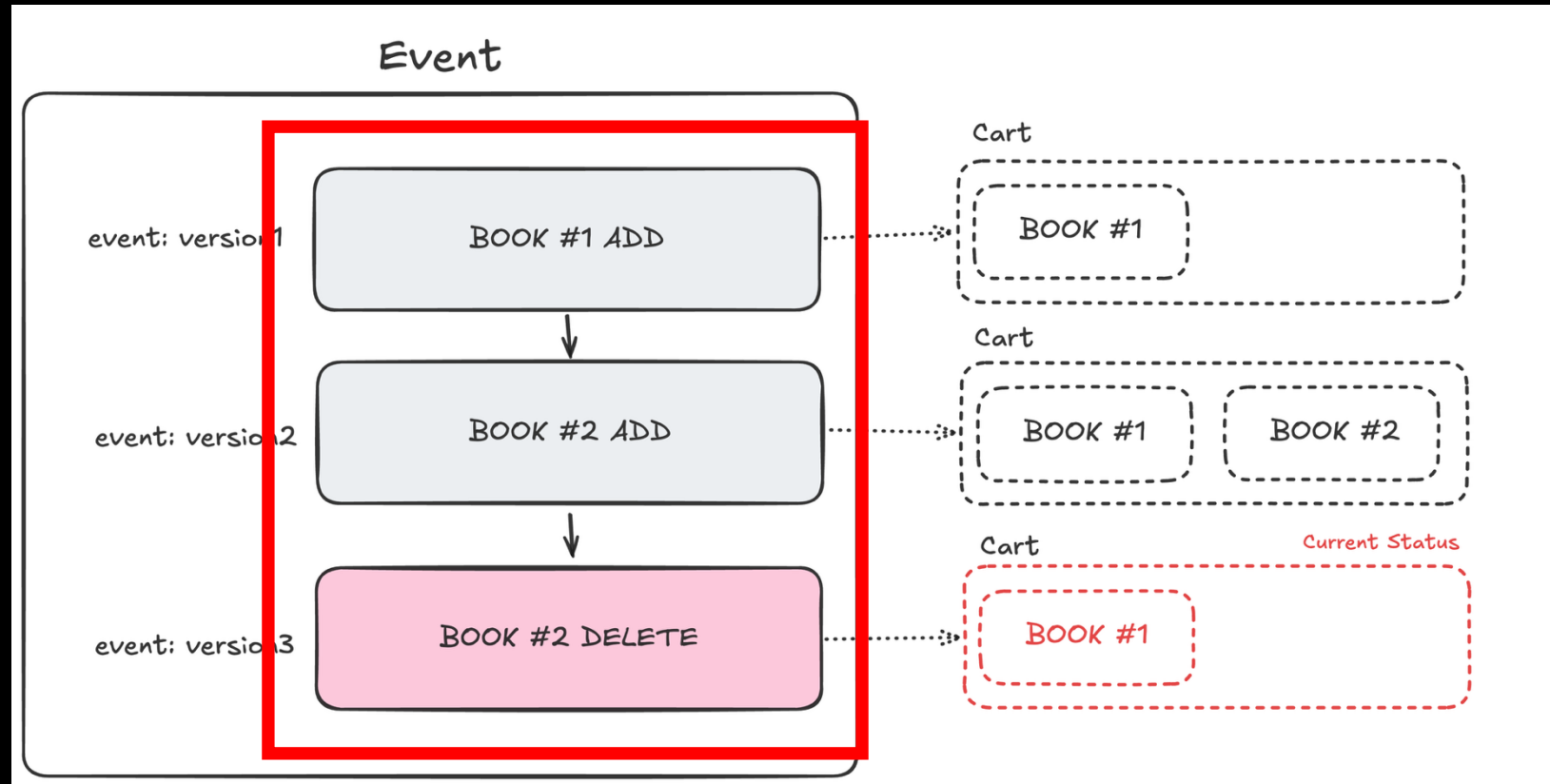
1. 이벤트 소싱이란?

상태가 아닌 사건을 저장하는 방식

어플리케이션의 모든 상태 변화를
순서에 따라 이벤트로 보관하는 저장 패턴

1. 이벤트 소싱이란?

예시 살펴보기



2. 이벤트의 구성 요소

이벤트는 불변하다. 오로지 추가(Append)만 가능하다.

Event의 특징

1. 불변하다.
2. 그렇기에 오로지 Append만 가능하다.
3. UPDATE나 DELETE의 개념이 없다.
(객체의 상태가 삭제되는 것 또한 “삭제되었다는 이벤트의 추가”일 뿐이다.)

2. 이벤트는 어디에 저장되는가?

영구 저장소인 EventStore에 저장된다.

EventStore

이벤트를 저장하는 저장소는 RDBMS, NoSQL, FILE 무엇이든 상관없다.
다만 반드시 영구 저장소에 저장되어야 한다.

3. 이벤트를 저장함으로서 얻는 이점

1. 과거의 이력 정보를 한번에 관리할 수 있다.

별도의 이력 테이블 생성 불필요

board		보드판 STATUS		
아이디	id	long	bigint	
게임아이디	game_id	long	bigint	
기물 종류	piece_type	string	varchar	
기물 팀	piece_team	string	varchar	
위치 행	board_row	int	bigint	
위치 열	board_column	int	bigint	

board_history		보드 이동 이력 테이블		
아이디	id	long	bigint	
게임아이디	game_id	long	bigint	
순서	order	long	bigint	
출발지 행	from_row	long	bigint	
출발지 열	from_column	long	bigint	
도착지 행	to_row	long	bigint	
도착지 열	to_column	long	bigint	

3. 이벤트를 저장함으로서 얻는 이점

2. DB에 쓰기 성능이 올라간다.

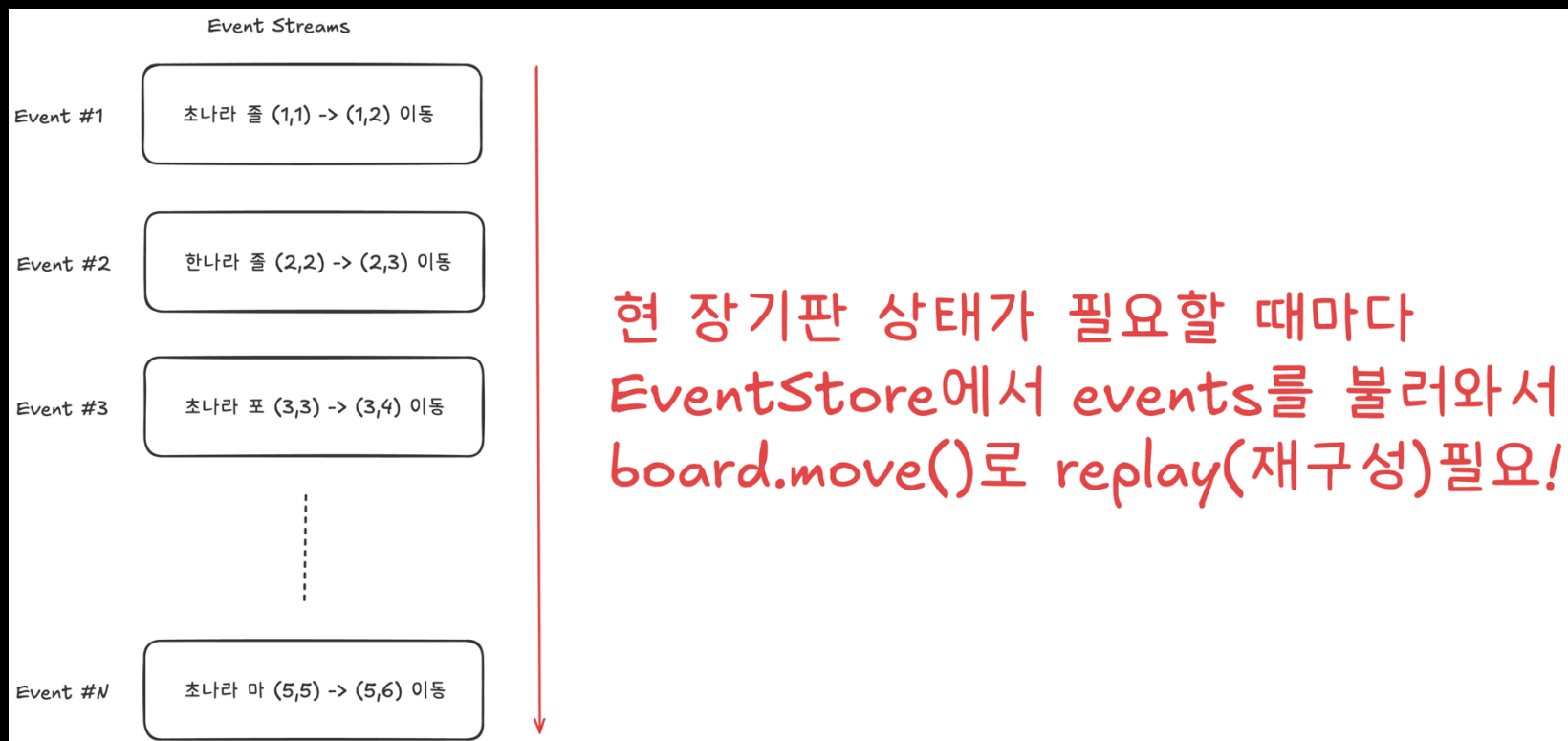
쓰기 작업은 오직 **이벤트 Append-Only** 작업이다.

기존 데이터를 읽거나 수정하는 **DELETE/UPDATE가 없기에** 쓰기 성능이 올라간다.

4. 현재 상태는 어떻게 재생하는가?

발생한 이벤트를 순차적으로 재생시키는 Replay

Replay는 어플리케이션 메모리에서 실행



4. 현재 상태는 어떻게 재생하는가?

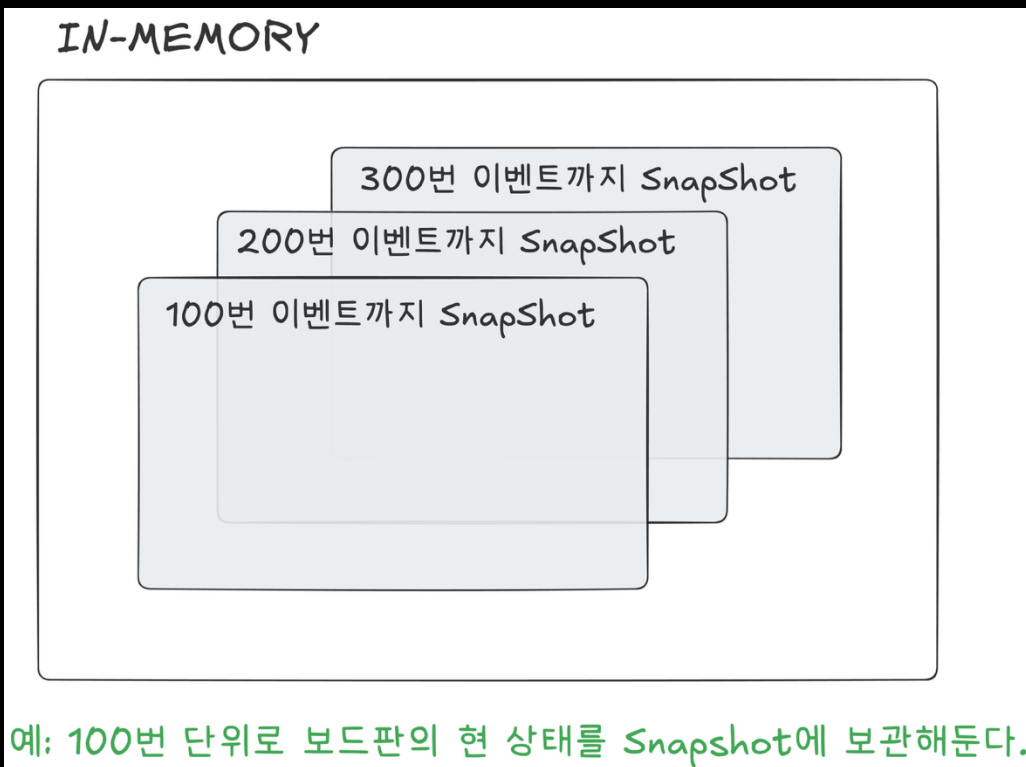
100개의 게임들이 동시에 실행되는 과정에서, 각 게임이 100만번 이벤트를 실행시켰다면?

각 게임당 기물 이동 “move 메서드”를 실행시키기 전에
총 100만번의 재생 연산이 필요

즉, 100×100 만번의 move 명령 수행 필요

5. 스냅샷이 필요한 이유

Replay 재생에 소요되는 작업을 줄이기 위해



스냅샷 생성시점 :
정책 별로, **이벤트 저장 시점**에 생성

스냅샷 저장 위치:
In-Memory에 저장하는 것이 성능상 유리

6. 읽기에서도 재구성이 필요할까?

이벤트 소싱의 단점

메모리 위에서 재생하기 위한 처리 시간이 병목

만약 “현재 종료된 모든 게임을 조회하라”라는 읽기 요청이 들어왔다면?

모든 게임을 Replay해야 한다....



Event Store와 독립적인 읽기 전용 모델을 따로 분리하자

7. CQRS란 무엇인가?

Command와 Query

Command: 상태를 변경하는 쓰기 요청 (예를 들어, 기물 이동 요청)

Query: 상태 값을 조회하는 읽기 요청 (예를 들어, 현재 턴 정보 조회)

7. CQRS란 무엇인가?

시스템 단위의 패턴

시스템 단위에서 읽기 모델과 쓰기 모델을 구분하자!

이벤트 소싱에서 CQRS를 도입하는 이유? 읽기 성능을 올리기 위해!

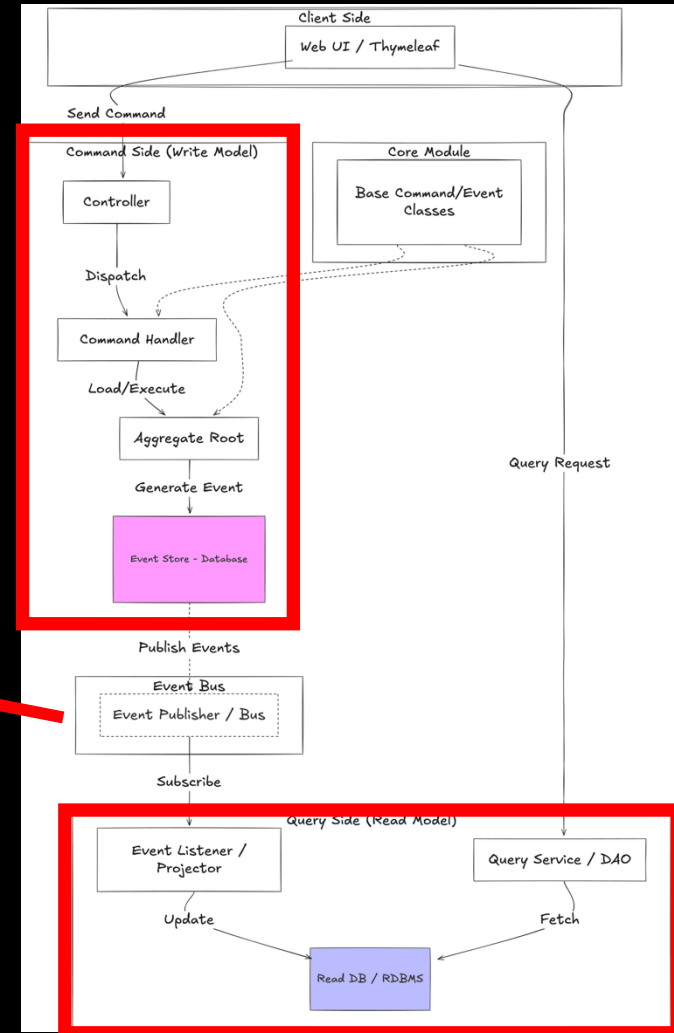
8. CQRS 적용시 전체 아키텍처

여기서 Write-Model을 주의 깊게 살펴보자.

쓰기 모델 전용 DB -> EVENT STORE

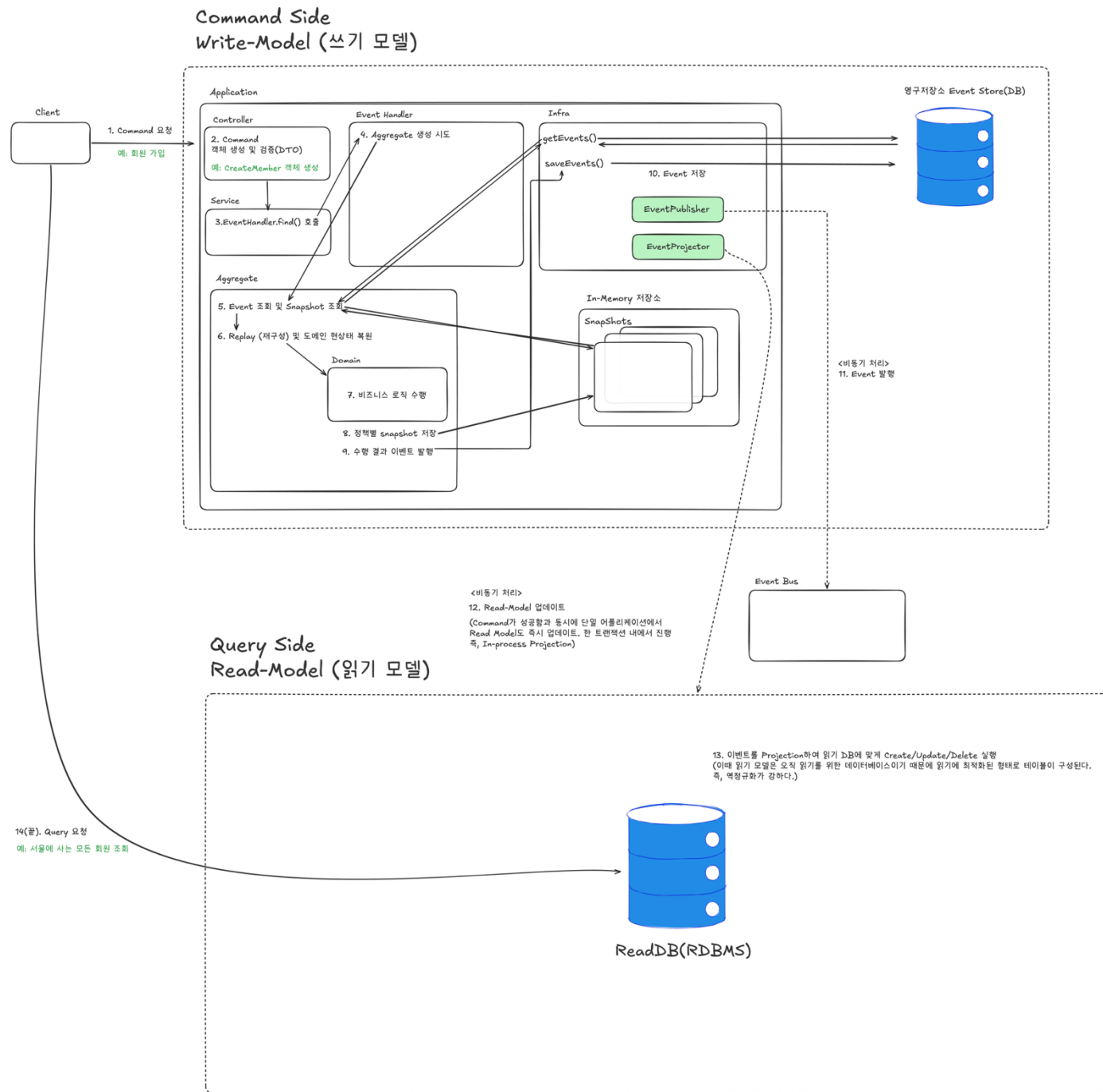
읽기 서버 갱신 비동기처리

읽기 모델 전용 DB -> 역정규화 RDBMS



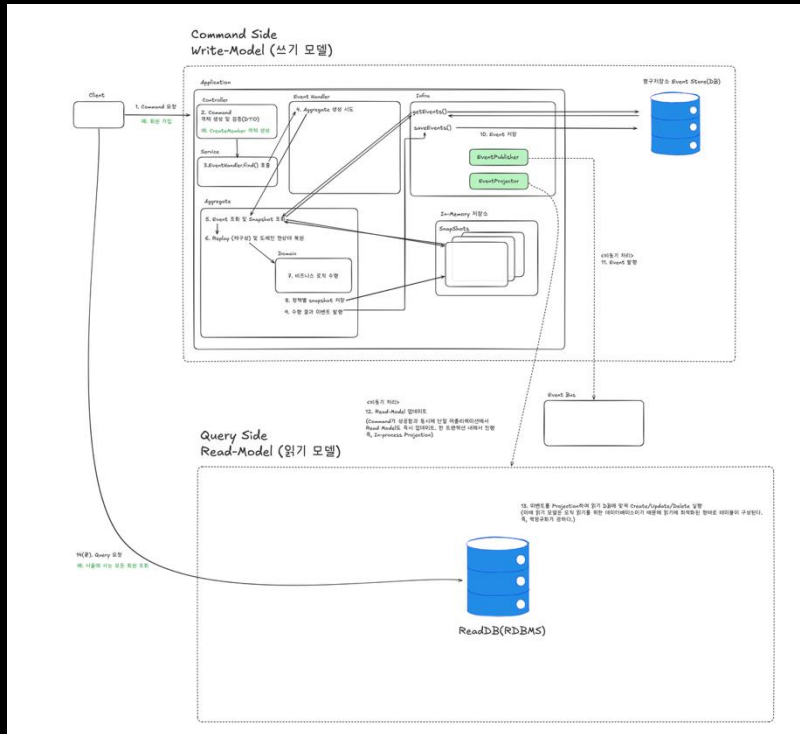
8. CQRS

내부 구현 사항



8. CQRS 적용시 전체 아키텍처

내부 구현 사항



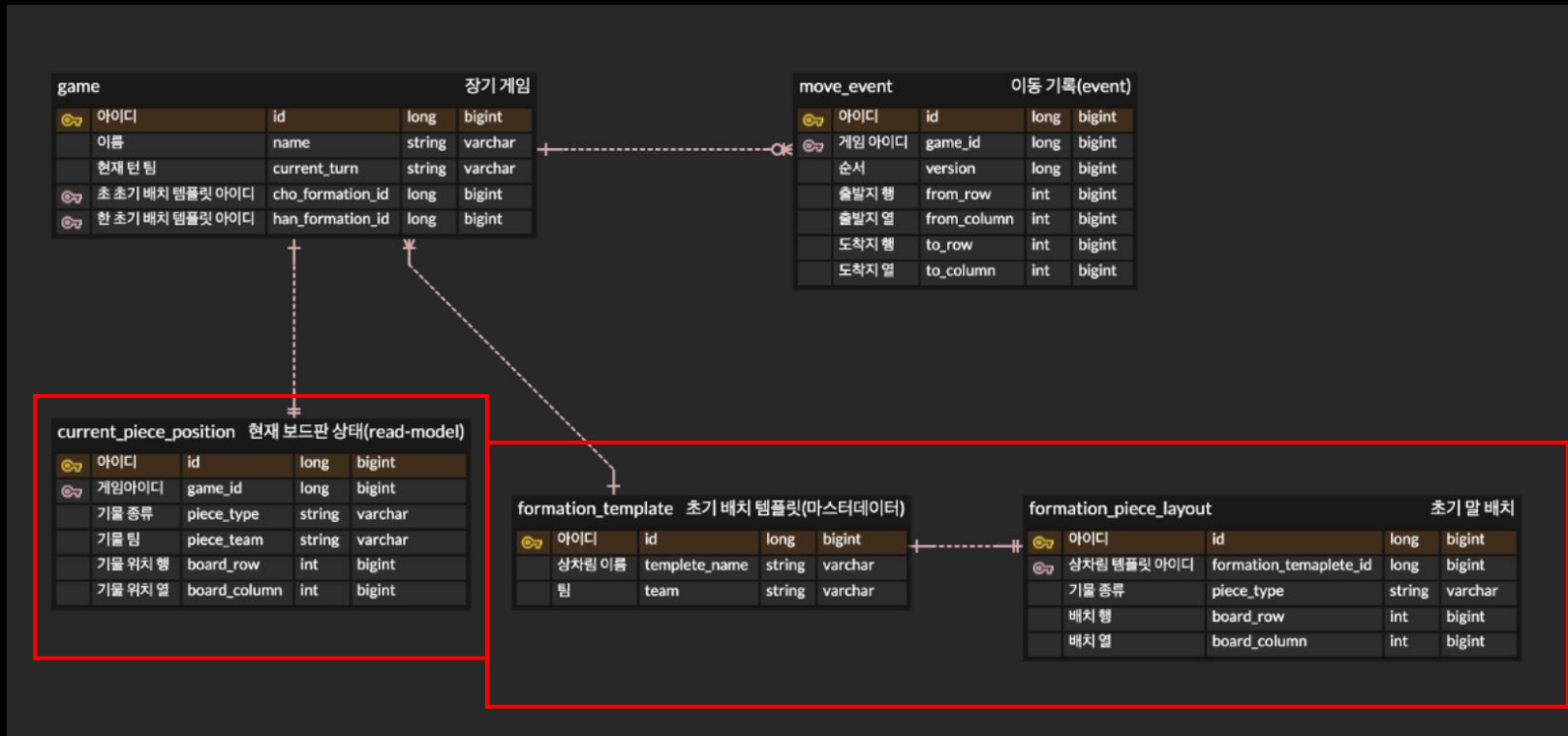
1. Command와 Query요청의 진입 모델은 다르다.
그리고 재구성은 쓰기 모델에서만 한다.

2. 이벤트가 생성되는 시점에, 한 트랜잭션에서 이벤트 저장,
(정책상 부합하면) 스냅샷 생성,
(비동기로) 읽기 모델 최신화를 진행한다.

3. 일반적으로 읽기 모델은 우리가 아는 익숙한 RDBMS 형태이다.
대신 읽기에 최적화된 역정규화 테이블이다.

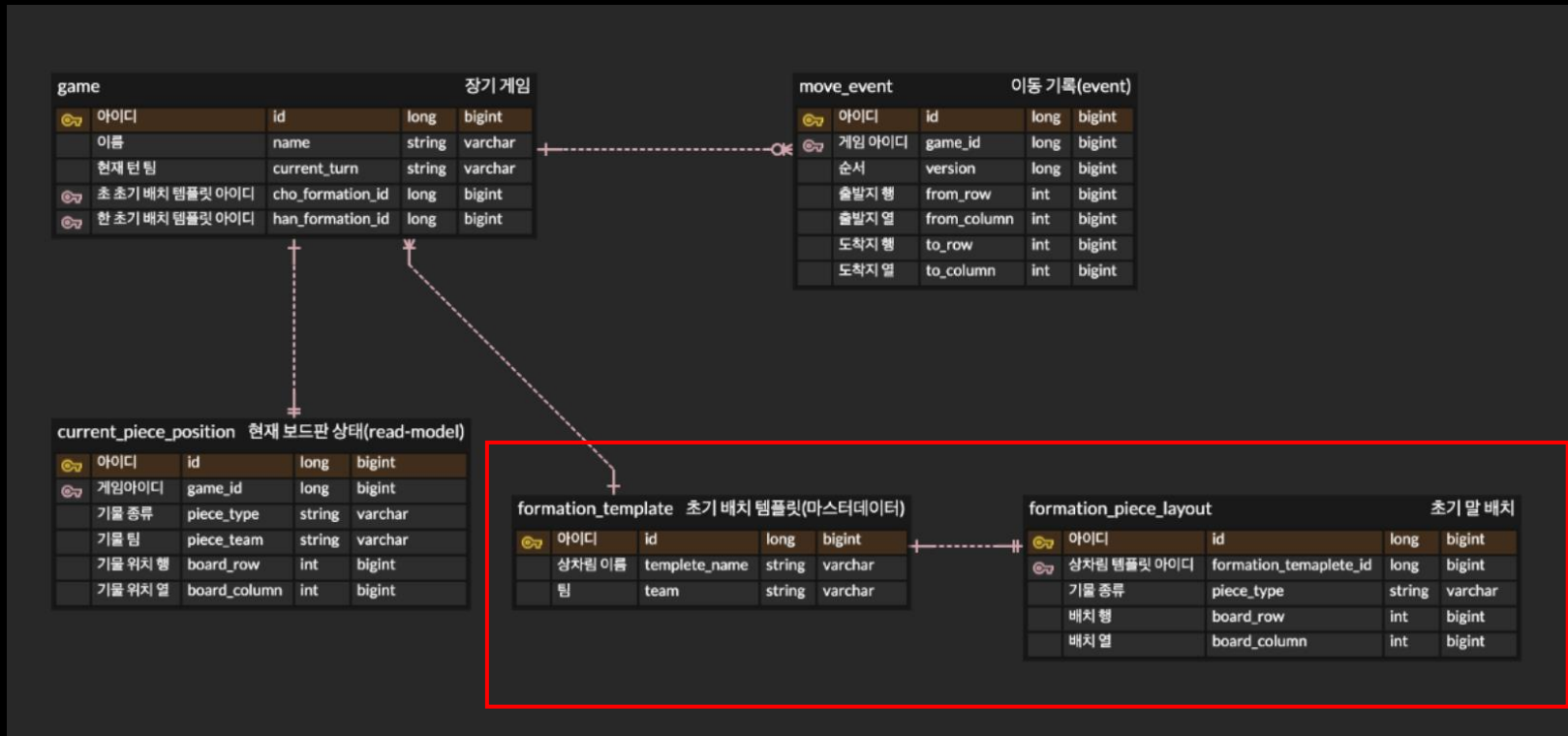
9. 장기와 어울리지 않는 이유

장기 구현시 ERD 구성도



9. 장기와 어울리지 않는 이유

1. 전략 수정 가능성을 대비해, 메타데이터 저장 테이블 필요



보드판 초기화 이벤트

```
{
  "eventId": "evt_20260426_001",
  "eventType": "JanggiBoardInitialized",
  "aggregateId": "game_room_777",
  "version": 1,
  "timestamp": "2026-04-26T12:00:00.000Z",
  "correlationId": "cmd_init_abc123",
  "payload": {
    "gameId": "game_room_777",
    "choPlayer": "user_id_kim",
    "hanPlayer": "user_id_lee",
    "choFormation": "MA_SANG_MA_SANG",
    "hanFormation": "SANG_MA_SANG_MA",
    "initialBoardState": {
      "positions": [
        {"pos": "0,0", "piece": "HAN_CHA"},
        {"pos": "0,1", "piece": "HAN_SANG"},
        {"pos": "0,2", "piece": "HAN_MA"},
        {"pos": "0,3", "piece": "HAN_SA"},
        {"pos": "0,5", "piece": "HAN_SA"},
        {"pos": "0,6", "piece": "HAN_MA"},
        {"pos": "0,7", "piece": "HAN_SANG"},
        {"pos": "0,8", "piece": "HAN_CHA"},
        {"pos": "1,4", "piece": "HAN_GUNG"},
        {"pos": "2,1", "piece": "HAN_PO"},
        {"pos": "2,7", "piece": "HAN_PO"},
        {"pos": "3,0", "piece": "HAN_ZOL"},
        {"pos": "3,2", "piece": "HAN_ZOL"},
        {"pos": "3,4", "piece": "HAN_ZOL"},
        {"pos": "3,6", "piece": "HAN_ZOL"},
        {"pos": "3,8", "piece": "HAN_ZOL"},
        {"pos": "9,0", "piece": "CHO_CHA"}
      ]
    }
  }
}
```

CHO

기물 이동 이벤트

```
{
  "eventId": "evt_20260426_002",
  "eventType": "JanggiPieceMoved",
  "aggregateId": "game_room_777",
  "version": 2,
  "timestamp": "2026-04-26T12:02:15.000Z",
  "correlationId": "cmd_move_xyz456",
  "payload": {
    "gameId": "game_room_777",
    "player": "CHO",
    "piece": "CHO_MA",
    "from": {"row": 9, "col": 2},
    "to": {"row": 7, "col": 3},
    "isCapture": true,
    "capturedPiece": "HAN_ZOL",
    "isCheck": false
  }
}
```

9. 장기와 어울리지 않는 이유

3. 장기 어플리케이션은 단일 사용자 콘솔 어플리케이션이다..

요청의 진입점이 단하나이며, 사용자도 단 한명



동시성 문제나, 읽기 쓰기 분리 불필요

게임 자체에서 이력으로 관리되어야하는 포인트



보드판 이동 이력만 필요

이벤트를 재구성하고 스냅샷을 관리해야 하는 이벤트 소싱의 비용



단일 서버기 때문에 관리포인트만 증가

마지막으로

발표 내용을 정리한 블로그 공유



[테크살롱] 이벤트 소싱과 CQRS

<https://junsangcho.tistory.com/25>

레퍼런스

SpringCamp2017 - Implementing EventSourcing & CQRS (구현부)



<https://youtu.be/12EGxMB8SR8?si=XfgsfuM8T6k7RTTp>